Alma Mater Studiorum · Università di Bologna

Second Cycle Degree
Artificial Intelligence

Combinatorial Decision Making and Optimization

Student:
Matteo Canghiari

Academic Year 2025/2026

Chapter 1

Introduction

Chapter 2

Modelling in CP

We need to start from the **model**: it doesn't just give us a formal representation of our problem, but also the modelling choices directly impact the search performance.

2.1 Formalization as a Constraint Satisfaction Problem (CSP)

A formal definition of any **Constraint Satisfaction Problem** (*CSP*) can be described as follows.

Definition

A CSP is a triple $\langle \mathbf{X}, \mathbf{D}, \mathbf{C} \rangle$ where:

- **X** is a set of decision variables $\{X_1, \ldots, X_n\}$, which we have to associate a value.
- **D** is a set of domains $\{D_1, \ldots, D_n\}$, where D_i is the domain of the decision variable X_i with $i \in \{1, \ldots, n\}$.
- **C** is a set of constraints $\{C_1, \ldots, C_m\}$. Every constraint C_i with $i \in \{1, \ldots, m\}$ is a mathematical relation over a subset of variables $\{X_j, \ldots, X_k\}$, denoted as $C_i\{X_j, \ldots, X_k\}$. Being a relation means a subset of the cartesian product of all the domains of the decision variables.

Any CSP would be described by this triple $\langle X, D, C \rangle$. A **solution** of a Constraint Satisfaction Problem is an assignment of values to the decision variables which satisfies all the constraints at the same time.

2.2 Constraint Optimization Problem

Previously, we talked about **satisfaction**: since we have constraints, the solution must satisfy all of them simultaneously. Anyway, sometimes we need to deal with **optimization**, that it's the same thing as before but there is an additional term, named **objective function**.

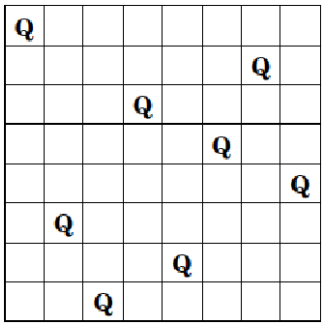
In this case, the formal definition of a **Constraint Optimization Problem** is described also by f, the formalization of the optimization criterion as an objective function. Therefore, our original triple becomes $\langle$**X,D,C,f**$\rangle$, where:

- **X** set of decision variables.
- **D** set of domains of the decision variables.
- **C** set of constraints applied to the decision variables.
- **f** formalization of the optimization criterion. It can be a **minimization objective** (*minimize* f) or a **maximization objective** (*maximize* $-f$).

Let's take a look at the following problems, understanding what decision variables, domains and constraints are really.

N-Queens

e.g. Place n queens in an $n \times n$ board so that no two queens can attack each other.



There are many ways to model this problem, however we try to solve it using the obvious approach. Our goal is to place n queens in a such way there is only one of them for each row, column and diagonal. We begin by defining the decision variables and their domains.

Decision variables and domains.

- Since each row must contain exactly only one queen, we define n decision variables $[X_1, X_2, \ldots, X_n]$; mapping one variable at each row we can satisfy the *no row attack* condition.

- The domain values $\{1, \ldots, n\}$ represent the available columns. An assignment $X_i = j$ indicates that queen in row i is positionated in column j.

To ensure no two queens attack each other, we must address three types of constraints:

1. *Row attacks.*
2. *Column attacks.*
3. *Diagonal attacks.*

Our modelling choice about decision variables allows us to satisfy the row constraint by design. Because each X_i represents a distinct row, we only need to explicitly define constraints for the remaining two.

Constraints.

- **alldifferent**$([X_1, X_2, \ldots, X_n])$.
- for all $i < j$ $|X_i - X_j| \neq |i - j|$.

These constraints prevent column and diagonal attacks, respectively. Specifically, the second constraint ensures that no two queens occupy the same diagonal, which occurs when the distance between their colums is equal to the distance between their rows.

Sudoku

e.g. Sudoku.

	6		1		4		5	
		8	3		5	6		
2								1
8			4		7			6
		6				3		
7			9		1			4
5								2
		7	2		6	9		
	4		5		8		7	

Solving the famous sudoku problem, we can do something similar to the previous example. Even in this problem, we get some decision making processes subject to constraints. As before, we begin by defining the decision variables.

Decision variables.

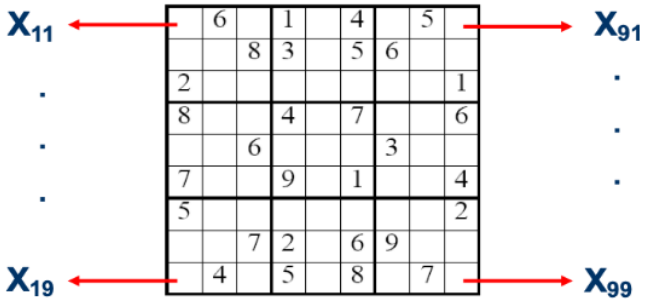
We start modelling the decision variables, which are in this case the **empty cells**: for every cell we have to give a decision. Anyway, focusing only on the empty cells we cannot model properly the given problem; also, the already

assigned cells can somehow influence our decisions.

So, what do we do?
Simply, in our model these variables will be assigned to the current values shown in the image.

Domains.
The domain values are any value in the set $\{1, \ldots, 9\}$.

Summing up, we can create 9×9 different decision variables, where each of them has a domain $\{1, \ldots, 9\}$; except for those variables that are already associated to a value.



Constraints.
- **alldifferent**$([X_{11}, X_{21}, X_{31}, \ldots, X_{91}])$.
- **alldifferent**$([X_{11}, X_{12}, X_{13}, \ldots, X_{19}])$.
- **alldifferent**$([X_{11}, X_{21}, X_{31}, X_{12}, X_{22}, X_{32}, X_{13}, X_{23}, X_{33}])$.
- Initial assignments, already described in the image (*for instance,* $X_{21} = 6$).

The first three constraints ensure different values for any row, column and box of the sudoku table.

Map colouring

e.g. Optimal map colouring.

Our goal is to colour regions such that neighboring regions are assigned different colors, using the minimum number of colors.

Decision variables and domains.
Every decision variable would be a region and any single domain value represents a different color. More formally, X_i with $i \in \{1, \dots, n\}$ are the decision variables, while $\{1, \dots, n\}$ it's the domain.

Since we have n regions, we could use n different colors satisfying the map colouring problem. However, we want to find out the minimum number of colors that we can use.

Constraints.
- $X_i \neq X_j$ for each neighboring region i and j.

We specify inside the constraint the necessary condition for stating when a region is a neighbor of an other one, since not all the regions are consecutive between each other.

Objective variable.
- $f = max(X_i)$
  where:
  - $X_i \rightarrow$ number of colors used by the map colouring
  - $max(\dots) \rightarrow$ function used to detect how many colors were used

Objective.
In order to obtain the minimum number of colors for the map colouring problem we have to minimize the objective function f.

2.3 Task scheduling

Constraint Programming is really powerful when we have to deal within **scheduling problems**. Typically, such problems are described by the following definition.

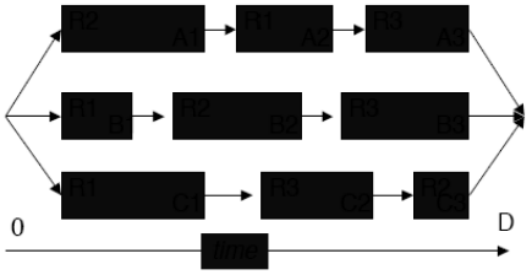
Task scheduling definition

Schedule n tasks on a machine, in time D, by obeying the temporal and precedence constraints:
1. Each task t_i has a specific fixed processing time p_i.
2. Each task t_i can be started after its release date r_i and must be completed before its deadline d_i.
3. Tasks cannot overlap in time.
4. Precedence relations must be respected.

Example

e.g. Task scheduling general problem.

Before we get into the solution, let's take a look at the image below.



Observing the figure, we can state:

- Every black box is a task t_i and its size determines the fixed processing time p_i.
- Every flow does not overlap tasks in the same row.
- Every flow respects the precedence relationships.

We could express decision variables, domains and constraints of any kind of task scheduling problem following a generic approach.

Generic approach.
Since the more interisting part of the optimization problem is the starting time of tasks t_i, the same decision variables can be directly described as the starting time; this is possible since we already know how much time any process requires.

Decision variables and domains.
We call $Start_i$ the decision variables representing the starting time of tasks t_i, where each of them takes a value from $[0 \dots D]$.

The domain $[0 \dots D]$ defines all the time we have available to compute our tasks t_i.

Constraints.

- Respect of release date and deadline.
  - for all $i \in \{1, \dots, n\}$ such that $r_i \leq Start_i \leq d_i - p_i$
    where:
    * $r_i \rightarrow$ release time
    * $d_i \rightarrow$ deadline time
    * $p_i \rightarrow$ processing time
    * $Start_i \rightarrow$ starting time of the task t_i

- No overlap in time.
  - for all $i < j \in \{1, \ldots, n\}$ such that $(Start_i + p_i \leq Start_j) \vee (Start_j + p_j \leq Start_i)$
    where:
    * $p_i \rightarrow$ processing time of the task t_i
    * $p_j \rightarrow$ processing time of the task t_j
    * $Start_i \rightarrow$ starting time of the task t_i
    * $Start_j \rightarrow$ starting time of the task t_j
- Precedence constraints.
  - $Start_i + p_i \leq Start_j$ for each pair of tasks $t_i \mapsto t_j$
    where:
    * $p_i \rightarrow$ processing time of the task t_i
    * $Start_i \rightarrow$ starting time of the task t_i
    * $Start_j \rightarrow$ starting time of the task t_j
    * $t_i \mapsto t_j \rightarrow$ precedence relation between tasks t_i and t_j

Final mark.
Before moving on, let's take a look at the final two constraints.

2nd No overlap in time.
The starting and ending time of any pair of tasks (t_i, t_j) must be either the same time or ordered time, but they are never allowed to overlap each other.

3rd Precedence constraints.
In almost of the cases, the second part of the previous constraint doesn't even exist ($\vee$ $(Start_j + p_j \leq Start_i)$), since in general the output of a task it's used by another task as input, relying on the precedence relationships.

2.4 Constraints

...